

010: StepPPO 训练时间与资源消耗诊断

范围

本文定位 StepPPO-v1 WebShop run 相比 GiGPO 更慢的原因, 并区分 compute、memory、validation 与策略行为导致的开销。比较对象是 GiGPO seed 2026 与当前 StepPPO-v1 seed 2026 日志。

日志来源

- GiGPO: logs/webshop_gigpo_qwen25_15b_4gpu_paper_align_simple_20260530_114442.log, 完整到 step 250。
- StepPPO-v1: logs/webshop_step_ppo_qwen25_15b_4gpu_paper_align_simple_20260604_102927.log, 当前可用到 step 187。
- 解析脚本: VISULIZATION/plot_step_ppo_time_resource_diagnostics.py。
- 统计输出: VISULIZATION/data/step_ppo_time_resource_summary.txt。
- 诊断图: VISULIZATION/figures/fig_step_ppo_time_resource_diagnostics.png。

整体时间

non-validation training rows 的均值为:

Metric	GiGPO	StepPPO-v1	Ratio
timing_s/step	132.220	209.435	1.584
timing_s/gen	110.111	167.502	1.521
timing_s/old_log_prob	3.539	8.814	2.491
timing_s/ref	3.330	4.580	1.376
timing_s/update_actor	14.551	27.615	1.898
perf/throughput	2252.975	1836.456	0.815

平均来看, StepPPO 非验证 step 约为 GiGPO 的 $1.58\times$ 。在后期 steps 151–187, StepPPO 非验证 step 约为 GiGPO 的 $2.02\times$ 。

主因: Rollout Generation 变慢

最大绝对时间差来自 rollout generation。平均 generation time 每个 non-validation step 增加约 57.4 秒, 占总 step time gap 的大部分。后期 segment 中, generation time gap 达到约 87.0 秒。

这主要是策略行为导致的: StepPPO 生成更长、更容易截断、格式更差的 response:

Metric	GiGPO	StepPPO-v1	Ratio
response_length/mean	135.890	155.042	1.141
response_length/clip_ratio	0.004	0.026	5.825
global_seqlen/mean	289217.624	385207.740	1.332
episode/valid_action_ratio	0.978	0.803	0.821

更长 response 和更多 clipping 会同时增加 vLLM generation、old log prob、ref log prob 和 actor update 的 sequence length 成本。

实现额外开销

StepPPO 确实存在直接实现开销。verl/workers/actor/step_ppo_actor.py 的 old-log-probability 路径在 value head 启用时不仅计算 token log probability，还会计算 old state values。actor update 路径也会重新计算 current values 并加入 clipped value loss。这解释了 old-log-probability $2.49\times$ 与 actor update $1.90\times$ 的开销。

另一个配置差异是 microbatch: StepPPO 4GPU 脚本使用 actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=4, GiGPO 脚本使用 8。更小 microbatch 降低显存压力，但增加 microbatch 数量，因此也会拖慢 update actor。

Validation 成本

validation rows 更贵，因为 testing 本身要执行环境交互。平均 validation-step timing 为

GiGPO : 395.122s, StepPPO : 548.947s.

平均 timing_s/testing 分别为 265.629 秒和 337.878 秒。因此比较 wall-clock 时必须保持 trainer.test_freq 一致。debug run 可以降低 validation 频率，但不能和 full protocol 的 wall-clock 直接比较。

资源消耗

当前不是 memory-bound。StepPPO 的平均 GPU allocated memory 和 CPU memory 反而低于 GiGPO:

Metric	GiGPO	StepPPO-v1
perf/max_memory_allocated_gb	71.710	60.905
perf/max_memory_reserved_gb	109.578	105.339
perf/cpu_memory_used_gb	551.620	433.726

这与 StepPPO 使用更小 actor microbatch 一致。主要瓶颈是 token generation 和额外 forward compute，而不是显存容量。

Wall-clock 粗估

按观测均值外推，完整 250-step StepPPO-v1 约需 19 小时；GiGPO seed 2026 约 13 小时。如果后期 generation quality 继续退化，StepPPO 实际成本可能更高。

建议

1. 先稳定 policy 行为。最大开销来自更长、更 invalid、更易 clipping 的 response。
2. 测试 `detach_value_backbone=True` 和更小 `value_loss_coef`，同时针对 stability 和 runtime。
3. 在显存允许时测试 StepPPO 的 `ppo_micro_batch_size_per_gpu=8`。
4. 优化 old-value collection，尽量把 log-probability 和 value computation 合并，避免额外 value forward。
5. 只在 smoke/debug 中降低 validation frequency；正式 wall-clock 对比必须保持一致。

结论

StepPPO 更慢有两个原因。主因是训练不稳定导致 rollout 行为变差，从而增加 response length、clipping 和 sequence length。次因是 value prediction/value loss 与较小 microbatch 带来的实现开销。当前日志显示它不是显存瓶颈。